



Algebra Relazionale

Prof Ionel Eduard Stan
A.A. 2025/2026

Contesto della lezione



1. Motivazioni e contesto
2. Modello relazionale (breve ripasso)
3. Categorie di operatori
4. Ogni operatore in dettaglio con esempi
5. Alberi di esecuzione ed equivalenze
6. Esercizi e riepilogo

Dal linguaggio SQL all'algebra relazionale



- **SQL è un linguaggio dichiarativo:** si specifica **cosa** vogliamo ottenere, non **come** ottenerlo
- **L'algebra relazionale è un linguaggio procedurale:** descrive esplicitamente i passi da eseguire tramite operatori su relazione
- Quando eseguiamo una query/interrogazione in SQL, l'**ottimizzatore di query** (in inglese, **query optimizer**) del DBMS la traduce in una corrispondente espressione in algebra relazionale (nascosta all'utente)
- **Conoscere l'algebra relazionale** aiuta a leggere i piani di esecuzione e a capire perché certe ristrutturazioni di query (push-down di filtri, join reordering...) migliorano le prestazioni

Cos'è l'algebra relazionale?

- 
- È un **linguaggio procedurale basato su concetti di tipo algebrico**: le operazioni sono definite su relazioni e producono a loro volta altre relazioni
 - Si compone di vari **operatori fondamentali**:
 - **Insiemistici**: unione ($\cup$), intersezione ($\cap$), differenza ($-$)
 - **Unari**: selezione (σ), proiezione (π), ridenominazione (ρ)
 - **Binari**: prodotto cartesiano ($\times$), theta-join ($\bowtie_{\theta}$), natural-join ($\bowtie$)
 - Permette di **comporre** operazioni in espressioni e alberi di esecuzione per interrogazioni anche molto complesse
 - Fornisce il **modello formale** su cui i DBMS si basano per tradurre e ottimizzare internamente le query SQL
 - Le **equivalenze** tra espressioni algebriche (es., push-down di σ/π) sono l'arma principale per produrre piani più efficienti

Modello relazionale: Strutture di base



- Una **relazione** è un'astrazione matematica che definisce un **insieme** di tuple, ciascuna definita su uno **schema** comune
- **Schema di relazione $R(X)$:**
 - R = nome della relazione
 - X = insieme di attributi $\{A_1, A_2, \dots, A_n\}$, ciascuno associato a un dominio di valori
- **Tupla:** istanza di R , ossia un vettore ordinato di valori $(v_1, \dots, v_n)$ in $R(X)$
- **Dominio:** insieme di valori leciti per ciascun attributo (es., interi, stringhe, date)
- **Cardinalità $|R|$:** numero di tuple in R (sempre finito nel DB reale)
- **Grado $|X|$:** numero di attributi di R
- **Tabella:** rappresentazione “intuitiva” di R , dove le righe sono le tuple e le colonne sono gli attributi

Modello relazionale: Chiavi e vincoli

- 
- **Superchiave:** insieme di attributi $K \subseteq X$ tale che non esistono due tuple distinte t_1, t_2 con $t_1[K]=t_2[K]$
 - **Chiave:** superchiave **minimale** (nessun sottoinsieme proprio di K è superchiave)
 - Esempio: {Matricola} minimale $\Rightarrow$ chiave primaria di STUDENTI
 - **Chiavi candidate:** tutte le chiavi possibili; una di queste diventa **primaria** (imposta UNIQUE + NOT NULL)
 - **Vincoli di integrità referenziale:**
 - Si dichiara FOREIGN KEY($A_1, \dots, A_n$) REFERENCES S($B_1, \dots, B_m$)
 - Impone che ogni valore non-NULL di ($A_1, \dots, A_n$) in R appaia in S su ($B_1, \dots, B_m$)
 - **Vincoli di tupla:** espressioni booleane su attributi
 - **Vincoli di dominio:** es. $0 \leq \text{Voto} \leq 30$

Categorie di operatori

- 
- L'algebra relazionale è un linguaggio procedurale costituito da un insieme di operatori che trasformano relazioni in relazioni
 - **Operatori insiemistici** (richiedono schemi identici):
 - **Unione ($\cup$)**
 - **Intersezione ($\cap$)**
 - **Differenza ($-$)**
 - **Operatori unari** (agiscono su una sola relazione):
 - **Ridenominazione (ρ)**: rinomina relazione e/o attributi per armonizzare schemi
 - **Selezione (σ)**: filtra le tuple che soddisfano una condizione (taglio orizzontale)
 - **Proiezione (π)**: estrae un sottoinsieme di attributi, eliminando duplicati (taglio verticale)
 - **Operatori binari** (combina due relazioni):
 - **Prodotto cartesiano ($\times$)**: accoppia ogni tupla di R con ogni tupla di S
 - **Theta-join ($\bowtie_{\theta}$)**: join con condizione generica
 - **Natural join ($\bowtie$)**: join automatico su attributi omonimi
 - **Operatori derivati** (costruiti via quelli fondamentali):
 - **Divisione ($\div$)**: usata per query universali ("tutti gli elementi che...")

Unione (U): Definizione

- 
- $R \cup S = \{ t \mid t \in R \text{ OR } t \in S \}$
 - È un'operazione insiemistica: elimina i duplicati
 - Richiede **schemi identici** (stessi attributi e domini)
 - In SQL corrisponde a UNION

Unione (U): Esempio

- **Relazioni:**

- $R: (ID, Nome) = \{ (1, Alice), (2, Bob) \}$
- $S: (ID, Nome) = \{ (2, Bob), (3, Claudia) \}$

- $R \cup S = \{ (1, Alice), (2, Bob), (3, Claudia) \}$

- In SQL:

- `(SELECT ID, Nome FROM R) UNION (SELECT ID, Nome FROM S)`

ID	Nome
1	Alice
2	Bob

R

ID	Nome
2	Bob
3	Claudia

S

ID	Nome
1	Alice
2	Bob
3	Claudia

R U S

Intersezione ($\cap$): Definizione

- 
- $R \cap S = \{t \mid t \in R \text{ AND } t \in S\}$
 - Trova tutte le tuple comuni
 - Richiede ancora **schemi identici**
 - In SQL corrisponde a INTERSECT

Intersezione ($\cap$): Esempio

- **Relazioni:**

- $R: (ID, Nome) = \{ (1, Alice), (2, Bob) \}$
- $S: (ID, Nome) = \{ (2, Bob), (3, Claudia) \}$

- $R \cap S = \{ (2, Bob) \}$

- In SQL:

- `(SELECT ID, Nome FROM R) INTERSECT (SELECT ID, Nome FROM S)`

ID	Nome
1	Alice
2	Bob

R

ID	Nome
2	Bob
3	Claudia

S

ID	Nome
2	Bob

$R \cap S$

Differenze (-): Definizione

- 
- $R - S = \{ t \mid t \in R \text{ AND } t \notin S \}$
 - **Non commutativa:** $R - S \neq S - R$
 - Consente di rimuovere tuple indesiderate
 - In SQL corrisponde a EXCEPT (o MINUS)

Differenze (-): Esempio

- **Relazioni:**

- $R: (ID, Nome) = \{ (1, Alice), (2, Bob), (3, Claudia) \}$
- $S: (ID, Nome) = \{ (2, Bob) \}$

- $R - S = \{ (1, Alice), (3, Claudia) \}$

- In SQL:

- `(SELECT ID, Nome FROM R) EXCEPT (SELECT ID, Nome FROM S)`

ID	Nome
1	Alice
2	Bob
3	Claudia

R

ID	Nome
2	Bob

S

ID	Nome
1	Alice
3	Claudia

R - S

Ridenominazione (ρ): Definizione

- 
- $\rho_{B1, \dots, Bn \leftarrow A1, \dots, An}(R)$ cambia **solo** i nomi degli attributi $A_i \rightarrow B_i$, lasciando inalterati i valori
 - Utile per:
 - preparare operazioni insiemistiche (schemi compatibili)
 - disambiguare auto-join
 - Se volessimo cambiare anche il nome della relazione in R' , allora $R' \leftarrow \rho(R)$ oppure $\rho(R) \rightarrow R'$
 - In generale, se vogliamo “salvare” il risultato intermedio di espressioni, usiamo $\rightarrow$ oppure $\leftarrow$; vedremo alcuni esempi più avanti

Ridenominazione (ρ): Esempio



Padre	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele

PATERNITÀ

Madre	Figlio
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

MATERNITÀ

Genitore	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

$$\rho_{\text{Genitore} \leftarrow \text{Padre}}(\text{PATERNITÀ}) \cup \rho_{\text{Genitore} \leftarrow \text{Madre}}(\text{MATERNITÀ})$$

Selezione (σ): Concetto & Sintassi

- 
- **Filtra** tuple in base ad una condizione F
 - Sintassi: $\sigma_F(R)$
 - Predicati:
 - atomici: $A \theta B$ oppure $A \theta c$, con $\theta \in \{=, \neq, >, <, \geq, \leq\}$ è un operatore di confronto, A e B sono attributi di su cui il confronto θ abbia un senso, e c è una costante compatibile con il dominio di A
 - combinati con $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT)

Selezione (σ): Semantica



- Mantiene **stesso schema** di R
- Risultato: sottoinsieme delle tuple (o righe) di R
- Riduce la cardinalità, dunque abbiamo un'ottimizzazione se la spingiamo in anticipo (cioè, abbiamo meno tuple su cui applicare la sotto-operazione)

Selezione (σ): Esempio



ID	Età	Reparto
1	28	IT
2	35	HR
3	42	IT

R

ID	Età	Reparto
2	35	HR
3	42	IT

$$\sigma_{\text{Età} > 30}(R)$$

ID	Età	Reparto
3	42	IT

$$\sigma_{\text{Età} > 30 \wedge \text{Reparto} = \text{'IT'}}(R)$$

Proiezione (π): Concetto & Sintassi

- 
- **Seleziona** un sottoinsieme di attributi (colonne)
 - Sintassi: $\pi_{A_1, \dots, A_n}(R)$
 - Elimina i duplicati (semantica insiemistica)

Proiezione (π): Semantica

- 
- Schema ridotto a $\{A_1, \dots, A_n\}$
 - Se $\{A_1, \dots, A_n\}$ include una superchiave chiave, non si perdono tuple

Proiezione (π): Esempio



ID	Età	Reparto
1	28	IT
2	35	HR
3	42	IT

R

Reparto
IT
HR

$\pi_{\text{Reparto}}(R)$

Prodotto cartesiano (*): Definizione

- 
- $R \times S = \{ (t,u) \mid t \in R \wedge u \in S \}$ (concatena tuple)
 - Schema: attributi di R unito agli attributi di S
 - Cardinalità: $|R| \cdot |S|$ (spesso molto grande)

Prodotto cartesiano (*): Esempio



ID	Cliente
1	Andrea
2	Barbara

R

Codice	Prodotto	Quantità
A	Scarpe	100
B	Magliette	200
C	Occhiali	300

S

ID	Cliente	Codice	Prodotto	Quantità
1	Andrea	A	Scarpe	100
1	Andrea	B	Magliette	200
1	Andrea	C	Occhiali	300
2	Barbara	A	Scarpe	100
2	Barbara	B	Magliette	200
2	Barbara	C	Occhiali	300

$R \times S$

Theta-join ($\bowtie_{\theta}$): Definizione

- 
- Il prodotto cartesiano ha di solito poca utilità, in quanto concatena tuple non necessariamente correlate dal punto di vista semantico
 - In effetti, il **prodotto cartesiano viene spesso seguito da una selezione** che centra l'attenzione su tuple correlate secondo le esigenze
 - $R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$, dove θ è una qualsivoglia condizione su attributi di $R \cup S$
 - Quando la condizione θ è una congiunzione di atomi di uguaglianza (con un attributo della prima relazione e uno della seconda), il theta-join viene chiamato **equi-join**
 - Il **self-join** si ha quando facciamo la join tra una relazione con se stessa

Theta-join ($\bowtie_{\theta}$): Esempio



Impiegato	Progetto
Rossi	A
Neri	A
Neri	B

IMPIEGATI

Codice	Nome
A	Venere
B	Marte

PROGETTI

Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	B	Marte

$\text{IMPIEGATI} \bowtie_{\text{Progetto=Codice}} \text{PROGETTI} = \sigma_{\text{Progetto=Codice}}(\text{IMPIEGATI} \times \text{PROGETTI})$

Natural-join ($\bowtie$): Definizione

- Join su **tutti** gli attributi con lo stesso nome
- Elimina automaticamente le colonne duplicate
- È un **equi-join implicito**
 - Esempio: $R(A,B,C)$ e $S(B,C,D) \Rightarrow R \bowtie S = \pi_{A,B,C,D}(R \bowtie_{B=B' \wedge C=C'}(\rho_{B',C' \leftarrow B,C}(S))$

Natural-join ($\bowtie$): Esempio



Impiegato	Reparto
Rossi	Vendite
Neri	Produzione
Verdi	Produzione

R

Reparto	Capo
Produzione	Mori
Vendite	Bruni

S

Impiegato	Reparto	Capo
Rossi	Vendite	Bruni
Neri	Produzione	Mori
Verdi	Produzione	Mori

$R \bowtie S$

Alberi di esecuzione: Concetto



- Un'espressione (di algebra relazionale) si rappresenta come un **albero**:
 - Le **foglie** sono relazioni di base (es., R, S)
 - I **nodi interni** sono operatori (σ , π , $\bowtie$, ...)
- Vantaggi:
 - Visualizza chiaramente l'ordine di applicazione degli operatori
 - Base per l'**ottimizzazione** (riscrittura, push-down, reorder)
- In DB2 l'**Explain Plan** mostra alberi di accesso analoghi a quelli di algebra relazionale

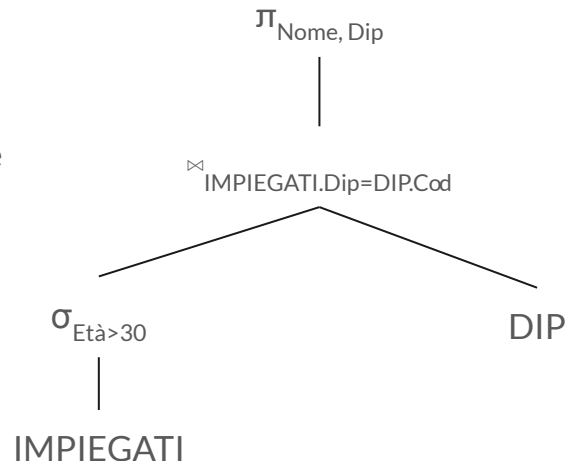
Alberi di esecuzione: Esempio



L'espressione $\pi_{\text{Nome, Dip}}(\sigma_{\text{Età} > 30}(\text{IMPIEGATI}) \bowtie_{\text{IMPIEGATI.Dip} = \text{DIP.Cod}} \text{DIP})$ ha il seguente albero di esecuzione

Commenti:

- Si applica il filtro $\sigma_{\text{Età} > 30}$ alla foglia IMPIEGATI
- Il join $\bowtie_{\text{IMPIEGATI.Dip} = \text{DIP.Cod}}$ produce tutte le tuple correlate
- La proiezione $\pi_{\text{Nome, Dip}}$ scarta tutti gli altri attributi



Precedenza e parentesizzazione

- 
- Convenzione di precedenza (senza parentesi):
 - σ, π, ρ (unari)
 - $\bowtie, \times$ (binari “forti”)
 - $\cup, \cap, -$ (insiemistici)
 - Esempio ambiguo:
 - $\pi_A(R \bowtie S) \cup T$
 - Si legge come $(\pi_A(R \bowtie S)) \cup T$
 - Se volessi prima unire R e T e poi fare la join, servono le parentesi: $\pi_A((R \cup T) \bowtie S)$

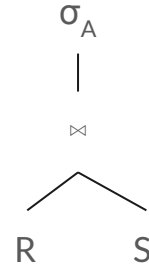
Alcune equivalenze algebriche

- 
1. **Atomizzazione delle selezioni:** $\sigma_{F_1 \wedge F_2}(R) \equiv \sigma_{F_1}(\sigma_{F_2}(R))$
 2. **Commutatività delle selezioni:** $\sigma_{F_1}(\sigma_{F_2}(R)) \equiv \sigma_{F_2}(\sigma_{F_1}(R))$
 3. **Selezione distributiva su $\cup$:** $\sigma_F(R_1 \cup R_2) \equiv \sigma_F(R_1) \cup \sigma_F(R_2)$
 4. **Push-down della selezione sul prodotto:** $\sigma_F(R \times S) \equiv \sigma_F(R) \times S$ se F solo su R
 5. **Push-down della selezione sul join:** $\sigma_F(R \bowtie S) \equiv \sigma_F(R) \bowtie S$ se F riferisce solo R (simmetrico per S)
 6. **Idempotenza/cascata di proiezioni:** $\pi_X(\pi_Y(R)) \equiv \pi_X(R)$ con $X \subseteq Y$
 7. **Commutazione π - σ :** $\pi_X(\sigma_F(R)) \equiv \sigma_F(\pi_X(R))$ se F usa solo attributi in X
 8. **Commutatività del join:** $R \bowtie S \equiv S \bowtie R$
 9. **Associatività del join:** $(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$
 10. **Commutatività/associatività di $\cup$:** $R \cup S \equiv S \cup R$; $(R \cup S) \cup T \equiv R \cup (S \cup T)$ (vale anche per l'intersezione)
 11. **Eliminazione del prodotto e selezione:** $\sigma_{R.A=S.B}(R \times S) \equiv R \bowtie_{R.A=S.B} S$

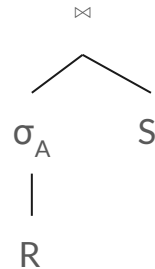
Equivalenza: Push-down di selezione



Si considerino le relazioni $R(A,B)$ ed $S(C,D)$, e l'espressione $\sigma_A(R \bowtie S)$. Il suo albero è



Che è equivalente a



Vantaggi:

- Riduce il costo del join (meno tuple da combinare)
- Base per l'ottimizzazione cost-based

Esercizio: Filtro, join, e proiezione

- Schema:

IMPIEGATO(CodImp, Cognome, Età, Stipendio, CodDip)

DIPARTIMENTO(CodDip, NomeDip, Telefono, Direttore)

- **Traccia:** Elencare Cognome degli impiegati con Stipendio > 3000€ insieme al NomeDip del loro dipartimento.
- **Soluzione:**

$$\pi_{\text{Cognome, NomeDip}}(\sigma_{\text{Stipendio} > 3000}(\text{IMPIEGATO}) \bowtie \text{DIPARTIMENTO}) \rightarrow \text{RISULTATO}(\text{CognI, NomeD})$$

- **Commenti:**
 - La selezione σ riduce $|\text{IMPIEGATO}|$ prima del natural join $\bowtie$ (su CodDip), dunque meno costi
 - La proiezione finale π restituisce gli attributi richiesti
 - Il risultato è una nuova relazione RISULTATO con due attributi (CognI, NomeD)

Esercizio: Catena di join



- **Schema:**

IMPIEGATO(CodImp, Cognome, CodDip)

PARTECIPAZIONE(CodImp, CodProg)

PROGETTO(CodProg, NomeProg, Budget)

- **Traccia:** Elencare Cognome e NomeProg per tutti gli impiegati che partecipano a progetti con budget > 150000€.

- **Soluzione:**

$$\pi_{\text{Cognome, NomeProg}}(\text{IMPIEGATO} \bowtie \text{PARTECIPAZIONE} \bowtie \sigma_{\text{Budget} > 150000}(\text{PROGETTO}))$$

- **Commenti:**

- La selezione σ è spinta prima di unirsi alla catena
- L'ordine dei join può essere riordinato dall'ottimizzatore (es., risultati intermedi più piccoli prima)

Esercizio: Self-join con rinominazione

- Schema:

IMPIEGATO(CodImp, Cognome, Età, CodDip)

- **Traccia:** Trovare le coppie Senior (età > 50) e Junior (età < 30) di impiegati dello stesso dipartimento
- **Soluzione:**

$$\rho_{\text{CodImpS,CognomeS,EtàS,CodDip}}(\sigma_{\text{Età}>50}(\text{IMPIEGATO})) \bowtie_{\text{CodDip}} \rho_{\text{CodImpJ,CognomeJ,EtàJ,CodDip}}(\sigma_{\text{Età}<30}(\text{IMPIEGATO}))$$

- **Commenti:**
 - La doppia rinominazione ρ disambigua i nomi e gli altri attributi
 - La condizione del natural join $\bowtie$ combina l'uguaglianza su CodDip
 - Le selezione σ filtra secondo le disuguaglianze specificate

Esercizio: Operatori insiemistici su sotto-insiemi

- Schema:

IMPIEGATO(CodImp, Cognome, Età, CodDip)

- **Traccia:** Trovare gli impiegati Junior (età < 30) e quelli Senior (età > 50). Costruire tre insiemi di CodImp:
 - a. Tutti gli impiegati che non hanno età tra 30 e 50 compresi.
 - b. Gli impiegati solo Junior (cioè, quelli con età < 30)
 - c. Impiegati sia Junior che Senior (cioè, sia quelli con età < 30 sia quelli con età > 50)
- **Soluzione:**

Junior $\leftarrow \pi_{\text{CodImp}}(\sigma_{\text{Età} < 30}(\text{IMPIEGATO}))$

Senior $\leftarrow \pi_{\text{CodImp}}(\sigma_{\text{Età} > 50}(\text{IMPIEGATO}))$

a. Junior $\cup$ Senior

b. Junior $-$ Senior

c. Junior $\cap$ Senior

- **Commenti:**
 - La c. dove restituire l'insieme vuoto $\emptyset$

Esercizio: Proiezione anticipata (equivalenza)

- Schema:

IMPIEGATO(CodImp, CodDip, Stipendio)

DIPARTIMENTO(CodDip, NomeDip, Telefono)

- **Traccia:** Dato il piano $\pi_{\text{NomeDip}}(\pi_{\text{CodImp, CodDip, NomeDip}}(\text{IMPIEGATO} \bowtie \text{DIPARTIMENTO}))$, si vuole un piano ottimizzato
- **Soluzione:**

$\pi_{\text{NomeDip}}(\pi_{\text{CodDip}}(\text{IMPIEGATO}) \bowtie \pi_{\text{CodDip, NomeDip}}(\text{DIPARTIMENTO}))$

- **Commenti:**
 - Una proiezione annidata π sugli stessi attributi è ridondante
 - Le due proiezioni interne π , una su IMPIEGATO e l'altra su DIPARTIMENTO, eliminano attributi non necessari prima di fare la join $\bowtie$

Esercizio: Natural join e theta-join

- **Schema:**

DIP(CodDip, NomeDip)

RESP(CodDip, CodImp) – Uno per dipartimento

IMPIEGATO(CodImp, Cognome, CodDip)

- **Traccia:** Recuperare NomeDip e Cognome del responsabile di ciascun dipartimento. Mostrare due soluzioni equivalenti: una con natural join e l'altra con theta-join
- **Soluzione:**

$\pi_{\text{NomeDip, Cognome}}(\text{DIP} \bowtie \text{RESP} \bowtie \text{IMPIEGATO})$

$\pi_{\text{NomeDip, Cognome}}((\text{DIP} \bowtie_{\text{DIP.CodDip=RESP.CodDip}} \text{RESP}) \bowtie_{\text{RESP.CodImp=IMPIEGATO.CodImp}} \text{IMPIEGATO})$

- **Commenti:**
 - Natural join è più compatto, ma usa **tutti** gli attributi omonimi
 - Theta-join esplicita i campi e resta valido se in futuro gli schemi cambiano

Riepilogo



- Oggi abbiamo visto:
 - **Motivazioni e contesto:** Perché la RA è il motore formale dietro SQL e i piani di esecuzione
 - **Breve ripasso del modello relazionale:** relazioni, schemi, chiavi, vincoli → base sintattica per l'algebra
 - **Categorie di operatori:** unari, binari, insiemistici
 - **Operatori in dettaglio (con esempi):** σ , π , ρ , $\cup$, $\cap$, $-$, $\times$, $\bowtie$ (theta & natural)
 - **Alberi di esecuzione ed equivalenze:** precedenza, push-down, commutatività/associatività del join, idempotenza $\pi, \times + \sigma \rightarrow \bowtie$, etc.
- Nella prossima lezione vedremo:
 - **Operatori avanzati:** divisione, semi-join, anti-join
 - **Estensioni della RA:** aggregazioni (γ), gestione valori NULL, bag semantics
 - **Da SQL ad algebra relazionale:** traduzione di query, sottoquery, viste
 - **Ottimizzazione cost-based:** stime di cardinalità, reorder bushy vs left-deep
 - **Ricorsione e Datalog (cenni):** oltre i limiti della RA classica